Informe desafío 1

Luis Andres Castillo Chicaiza
Facultad de Ingeniería
Universidad de Antioquia
Medellín, Colombia
luis.castillo1@udea.edu.co

Marcos Castrillón Velásquez
Facultad de Ingeniería
Universidad de Antioquia
Medellín, Colombia
marcos.castrillon@udea.edu.co

I. Contextualización

En este informe se describe el proceso de análisis y diseño para implementar una versión simplificada del juego Tetris.
La aplicación consiste en un motor de juego que funciona a través de una interfaz de consola, donde el usuario puede interactuar mediante el uso del teclado para mover y rotar las piezas del juego. Este juego no funciona inmediatamente, sino, mediante turnos en los que el programa muestra el estado actual del tablero, solicita una acción al usuario y actualiza el juego según el movimiento que realizó.
En este desafío se hace uso de la programación estructurada, incluyendo el uso de memoria dinámica, punteros y operaciones a nivel de bits.

II. Diseño de la solución

La solución planteada es utilizar datos y arquitectura de números simples sin signo, junto con arreglos, punteros y memoria dinámica.

- **Representación del tablero:** El tablero se representa mediante un doble puntero *uint8_t***, donde cada posición del primer nivel apunta a una fila del juego. A su vez, cada fila se almacena como un puntero simple, de tamaño *width/8*, ya que ancho del tablero debe ser múltiplo de 8. De esta manera, cada byte representa 8 celdas consecutivas y cada bit indica si una posición está ocupada o vacía.
- **Representación de las figuras:** Se modelan las cinco figuras mediante una matriz estática. Cada pieza y sus posibles rotaciones se codifican directamente en valores hexadecimales, esto, para almacenar su geometría como lógica de bits dentro de esta matriz 4x4.
- **Mecánica de colisiones:** En lugar de realizar validaciones celda por celda, se plantea el uso de operaciones a nivel de bits. Se calcula la posicion de la figura respecto a las fronteras de los bytes del tablero y, mediante desplazamientos lógicos, se alinean los bits de la pieza con los de la memoria.
- **Descenso de filas:** La validación de una fila llena se comprueba cuando todos sus bytes alcanzan el valor máximo (*0xFF*). Para eliminarla y hacer caer los bloques superiores, se reasigna las direcciones de memoria de las filas (*board[i] = board[i-1]*) y se hace el cambio.
- **Interacción y validación:** La captura de comandos y dimensiones se realiza mediante arreglos de caracteres (*char*), y las validaciones numéricas se procesan matemáticamente utilizando los valores de la tabla ASCII.

A. Arquitectura del sistema

En el siguiente esquema se presenta la arquitectura del juego (fig. 1), diseñada de manera modular y jerárquica de izquierda a derecha. Cada bloque está dividido en archivos fuente y de cabecera.

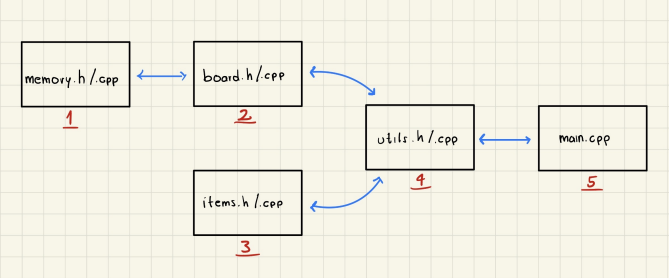

Fig. 1. Esquema del juego

1) ***memory.h / .cpp:*** Gestión de memoria. Se encarga de la reserva y liberación de la memoria dinámica utilizando punteros simples para las filas y punteros dobles para la matriz general.
2) ***board.h / .cpp:*** Librería del tablero. Depende del módulo de memoria para instanciar la cuadrícula. Contiene la lógica para ubicar coordenadas mediante índices de bytes. Contiene las operaciones para escribir, limpiar y leer los bits.
3) ***items.h / .cpp:*** Definición de figuras. Funciona como una base de datos independiente. Contiene la representación de las cinco figuras y sus rotaciones, las cuales están modeladas con valores hexadecimales.
4) ***utils.h / .cpp:*** Lógica. Interconexión de librerías. Calcula las colisiones mediante desplazamientos lógicos, además de validar datos ingresados por el usuario.
5) ***main.cpp:*** Bucle de juego. Inicializa valores, obtiene datos ingresados por el usuario y ejecuta el bucle principal del juego.

B. Flujo general de ejecución

El juego inicia con la validación de las dimensiones del tablero ingresadas por el usuario, procesando los caracteres ASCII de la entrada para garantizar que el ancho sea múltiplo de 8 y el alto válido. Una vez confirmados los parámetros, se reserva la memoria requerida para la matriz bidimensional mediante punteros y da inicio al ciclo del juego, instanciando la primera figura aleatoria desde el diccionario estático de piezas para posicionarla en el centro superior de la cuadrícula.

Durante el bucle principal, el sistema lee la tecla del jugador a través de la consola para determinar el desplazamiento o la rotación de la pieza. Antes de aplicar el movimiento, se calculan posibles colisiones con los bloques ya establecidos. Si la figura colisiona en su descenso, se fija permanentemente en la memoria del tablero y se evalua la eliminación de filas completas; luego, se genera una nueva pieza, repitiendo el proceso hasta que el límite superior se bloquee y finalice el juego.

III. Problemas que afrontamos

En el desarrollo del juego, se presentaron algunos retos.

- En una primera etapa se definió la estructura general del proyecto, con solo 2 o 3 módulos, pero no era ordenado ni eficiente, ya que se tenían módulos con procesos que eran mejor tenerlos diferenciados, así como variables o funciones necesarias para un módulo en específico.
- Con las piezas, en un inicio se pensó hacer de manera lógica las rotaciones, pero luego de tener problemas con esto, se hizo una matriz general las cuales tenían, sus figuras estándar así como sus rotaciones ya definidas para luego acceder a ellas de manera fácil.
- La colisión y fijación de piezas representó un problema, porque fue necesario evaluar el desplazamiento de bits entre filas para detectar superposiciones, así como también también fue un problema las figuras que se separaban entre dos posiciones de memoria contiguas, ya que requerían un mayor análisis para hacer los movimientos u operaciones.
- En el momento de hacer llamados de funciones en el main, cuando el usuario ingresaba la letra ”s” para bajar la figura, esta hacía un doble salto hacia abajo debido a un error en los cálculos para bajar la figura. Como solución en la sección donde se obtiene la instrucción del usuario, al presionar la ”s” se eliminó que la figura baje al presionarla, ya que esto lo hace por si solo con cualquier otra letra no definida en los controles.

A. Evolución de la solución y consideraciones

Como evolución desde el informe preliminar hasta la implementacion final se tiene que el código se ”modularizó” para hacer más eficiente y entendible.

Además es importante tener en cuenta las consideraciones de las reglas del juego, ya que en este trabajo no se hicieron validaciones para las instrucciones de movimiento y rotación, esto implica que si el usuario ingresa un dato diferente a: ”a”, ”d”, ”s”, ”r”, la figura bajará automáticamente para continuar el flujo del juego.

Finalmente, se adjunta enlace al repositorio GIT para acceder a los archivos fuente de este desafío: https://github.com/LuisC645/Desafio_1